Efficient Thinking II — An Evaluator × Search Pattern Across Games and Reasoning

Across games, language reasoning, and sequential control — measured on one calibrated scale

Louay Alsakka · 2026 · *working paper*

Abstract

Efficient Thinking I observed, in chess, that playing strength factors into two parts — **strength = evaluator × search**: a fixed evaluator (one forward pass) sets a base level, inference-time search multiplies it, and self-learning plateaus because the *learned evaluator* is the binding constraint. This paper asks whether that pattern is specific to chess or recurs elsewhere. We test it in three directions — a *simpler* solved game (Connect-4), a *non-game* domain (LLM mathematical reasoning), and a *sequential-control* MDP (a gridworld with an exact oracle) — and, to make numbers comparable across domains, we introduce **GELO**, a calibrated cross-domain capability scale (chess Elo, Bradley–Terry, and item-response theory are one logistic model). The decomposition transfers: search is a large, portable lever that scales with inference compute and then saturates against the evaluator's ceiling. Most sharply, in reasoning a perfect verifier breaks a self-consistency ceiling that more search cannot (**+14.2 points**), a graded verifier traces a smooth capability curve (**75% → 88%**), and across five self-improvement experiments the plateau never breaks — because self-play converges to its own level of play; climbing past it requires importing information (an external oracle). We also find a sharp cross-domain *contrast*: in reasoning, base-model **size dominates search** on the compute-efficiency frontier — the opposite of chess — which the same thesis explains (search extracts what a model already contains; a base too weak to solve leaves nothing to extract). We report these as recurring empirical patterns observed under deliberately modest compute (two Apple-Silicon machines), not as proven laws. **Across the domains and compute budgets studied here, one through-line is consistent: the evaluator — not parameters or search budget — is the binding constraint.**

1. Introduction

Modern game-playing and reasoning systems both gain most of their capability from two levers that are easy to conflate: a *learned evaluator* (a network's single-pass judgement) and *search* (spending inference-time compute to look further before committing). Efficient

Thinking I separated them in chess and found that a tiny 3.45M-parameter network reaches ~2150 Elo open-loop and ~2800 with MCTS, that search scales roughly log-linearly with simulations, and that self-play stalls because the evaluator — not parameters or search budget — is the ceiling.

If that structure is real rather than a chess artifact, it should reappear in very different settings. This paper puts it to three tests: **Connect-4**, a *solved* game where a perfect oracle and a graded opponent ladder let us measure everything exactly; **LLM mathematical reasoning**, a non-game domain with a natural verifier (a checkable final answer); and a **gridworld control MDP**, a sequential-decision setting where value iteration supplies an exact oracle. Our contributions:

1. **GELO** (§2): a single logistic capability scale on which a chess rating, a Connect-4 rating, and a reasoning ability are directly comparable, with a *calibrate-first* protocol and a goodness-of-fit gate.
2. **The decomposition transfers** (§3–§5): evaluator × search holds in Connect-4, in reasoning, and in control; the evaluator/search *balance* shifts with which side you have starved.
3. **The evaluator is the binding constraint, in every setting we test** (§4): a perfect verifier breaks a search-saturated ceiling (+14.2), verifier quality traces a continuous capability curve, and — a subtle asymmetry — search improves a *policy* but barely improves the *evaluator*.
4. **Self-improvement can't beat its own signal** (§5): five experiments across two games fail to break the plateau with self-generated targets; a single change — swapping in an external oracle — breaks it.
5. **A cross-domain contrast** (§4): in reasoning, base-model size dominates search on the compute-efficiency frontier — the opposite of chess — reconciled by the same principle.

2. GELO: one scale across domains

Capability is a latent ability θ on one logistic model that unifies chess Elo, Bradley–Terry, and item-response theory (Rasch): P(win / solve) = 1 / (1 + 10^(−(θ − d)/400)). We keep chess's constants (**400 GELO = 10× the odds; 120 GELO = one doubling**), so a chess GELO *is* a chess Elo, and add a *calibrate-first* protocol: build a reference ladder → fit ratings from a cross-table (not single win-rates) → **gate on logistic goodness-of-fit** → pin interpretable anchors → only then rate agents. An agent can be placed two equivalent ways: against a graded **reference ladder** (opponents or difficulty tiers), or **pairwise agent-vs-agent** with one agent pinned as the reference. Full spec in `docs/gelo.md`.

The first calibration, on Connect-4, passes the gate cleanly: mean |predicted − observed| pairwise win-rate = **0.058** — the logistic model genuinely fits the game, so ratings are earned

rather than assumed (scale anchored random := 0). And reasoning lands on the *same* axis via either route:

- **Pairwise ("beat another LLM").** Five models answer the same MATH problems; a master judge (Kimi-K2.5) scores every head-to-head → Bradley–Terry GELO, **anchored Kimi-K2.5 := 2800** (the "grandmaster" of the set), the small models placed below with headroom: **Qwen3.5-4B +2743 · Qwen2.5-1.5B +2562 · Qwen2.5-3B +2509 · Qwen2.5-0.5B +2297.** The 0.5B sits ~500 GELO below the frontier; mid-models bunch within noise (50 questions). The judge agrees with the ground-truth verifier on **72%** of decisive pairs — decent, but *itself evaluator-limited*: a referee can only rank as well as it can reason (which is also why, on checkable tasks, the verifier still beats an LLM judge). The anchor value is free; we choose 2800 so the numbers read like chess (elite ≈ 2800, room down toward a novice floor).
- **Difficulty-anchored (IRT).** The same machinery against MATH difficulty tiers gives a monotonic ladder (L1 +1273 → L5 +1712, ~+110 GELO/level) and places a model by which tier it half-solves.

3. Arm A — a simpler game (Connect-4)

Connect-4 is solved, so the exact solver is a perfect oracle and a depth-limited alpha-beta gives a calibrated opponent ladder. A small convolutional network (policy + value) is the evaluator; PUCT MCTS is the closed loop.

A calibrated opponent ladder. random 0 → **depth-1 +804** → depth-2…6 +954…+1070. The *first ply of tactics* is the single biggest step (+804) — larger than all five deeper plies combined — and the heuristic ladder then saturates: diminishing returns on search depth, quantified on a real axis.

Evaluator × search decomposition (12k oracle labels). Open-loop (raw net) **+644 GELO**, closed-loop (MCTS-200) **+880 GELO** → a **search lift of ≈ +236 GELO (~4× the odds per game)** — the same order as chess's search lift. The decomposition transfers to a second game.

The lever balance is data-dependent, not intrinsic. With only 12k labels the raw net loses even to 1-ply search, which reads as "search-dominated." But tracing the open-loop ceiling against data — **+642 (12k) → +747 (24k) → +798 (50k)** — the raw net reaches depth-1 parity (+804) by 50k. The apparent search-dominance was mostly a *starved evaluator*: fed data, it catches up to low-depth search. The honest statement is that the evaluator/search balance moves with how much you have spent on the evaluator — in either direction.

4. Arm B — reasoning (LLM)

The mapping from chess to language:

chess	LLM reasoning
position	partial reasoning trace
policy (move priors)	next-step distribution
evaluation: P(win)	**P(this reasoning is correct)** $\in [0,1]$
terminal result (win/loss)	**verifier on the final answer** (exact in math/code)
MCTS search	inference-time compute over reasoning paths
training the net	RL / fine-tune — RLVR is AlphaZero's loop in language

Two structural notes: the evaluator's output is a **P(correct)** — binary from a verifier, scalar from a reward model, vote-fraction from self-consistency — and a *well-calibrated* one is the open problem; and search has two axes — parallel (best-of-N, self-consistency) and serial (long chain-of-thought), the latter *internalized* into the policy by RLVR rather than kept external as in AlphaZero.

How search is implemented here — granularity and who evaluates. This is worth stating concretely, because an LLM is not a game engine. An LLM has **no built-in correctness-evaluator**: its only native scorer is the next-token softmax, which judges token *plausibility* (what word comes next), never answer *correctness*. Every search result below therefore operates at **whole-answer granularity** with an **external** evaluator, by a uniform procedure: sample N *complete* answers from the policy (temperature

> *0, so the N chains-of-thought actually differ), let each run to completion (hundreds of tokens), extract its final answer, then apply a **selector** over the finished set — majority vote (a verifier-free evaluator), a checkable verifier (perfect), or the Kimi-K2.5 judge (a real, imperfect model). We do **not** score or branch per token, nor per reasoning step — that would be tree-of-thought with a process-reward model, which we leave to future work. The two search knobs are thus **N** (how many answers) and the **selector** (how you pick), both entirely external to the frozen weights. This is the language analog of AlphaZero's split: the policy proposes a full line of play, and a separate value function judges it — and here, as there, the judge is the ceiling.*

Search scales accuracy, then saturates. Qwen-4B on GSM8K (120 problems, 1024-token budget): greedy pass@1 = **66.7%**; self-consistency@N = 69.2 (N=1) → 73.3 (4) → **77.5 (16)** → **77.5 (32)** — search buys **+8–11 points** (the analog of Elo-vs-sims) and then saturates by N=16. Majority vote is a *verifier-free* evaluator, and it stops helping.

The saturation is an *evaluator* ceiling, not a policy or search ceiling. From the same samples, self-consistency saturates at 77.5% while **oracle-best-of-N (a perfect verifier) climbs to 91.7% and is still rising at N=32** — an evaluator gap of **+14.2 points**. The right answer is *in the sample set* 91.7% of the time; consensus just can't select it. In language, exactly as in chess, the verifier is the binding constraint — and with a perfect evaluator, search keeps paying past where consensus stalls.

A graded verifier draws a continuous capability curve. Varying a verifier's per-item accuracy q, the resulting accuracy climbs smoothly from **75.0% at q=0.5 (verifier-free consensus) → 88.3% at q=1.0 (perfect verifier)** — 77.7 / 80.6 / 83.0 / 85.9 at q = 0.6/0.7/0.8/0.9, about +2.6 points per 0.1 of verifier accuracy. There is no threshold: *every* increment of a better evaluator buys capability. To go further, improve the evaluator.

A *real* evaluator — not just a perfect one — extracts more than consensus. Selecting among N Qwen-1.5B samples with the Kimi-K2.5 judge as an (imperfect) scorer beats majority vote at every N and captures most of the consensus→oracle headroom: at N=8, self-consistency **65.0%** → **Kimi-best-of-N 75.0%** → oracle **83.3%** (and at N=4, 61.7% → 73.3% → 81.7%). A stronger *selector* buys ~+10 points on the *same* samples — so "more search" pays far more when a better evaluator *spends* it than when majority vote does. This is the deployable form of the +14.2 result: to turn search into capability, improve the selector, not just raise N.

Search improves a policy but barely an evaluator. Running the master judge itself with self-consistency (majority of N judgments) raised agreement with the verifier only **72.5% → 75.0% (N:1→5)** — a ~+2.5-point nudge, versus the +8–11 points self-consistency buys a *policy*. The reason is instructive: a policy benefits because *some* of many attempts land on the answer and search selects them; but if a judge cannot reason a problem out, repeating the judgment reproduces the same *systematic* error. So you can partly buy back a weak *policy* with compute, but **not a weak *evaluator*** — which is exactly why the evaluator's quality, not its search budget, binds.

The serial axis (thinking length) saturates fast for a base model. Measuring the *other* search axis — greedy accuracy vs generation budget — Qwen-1.5B climbs 7.5% (128 tok) → 41.2 (256) → **48.8% (512), then flat** through 2048; Qwen-3B similarly 5.0 → 41.2 → **67.5% (512+), flat.** The gain is almost entirely a *don't-truncate* effect: a non-thinking base model finishes its chain of thought in ~512 tokens and stops, so extra budget buys nothing. Genuine o1/R1-style serial scaling requires a model *trained* to keep deliberating. So of the two search axes, **parallel (more samples, a better selector) is the live lever for an untrained base model, while the serial axis is capped at "enough room to finish"** — once more, search extracts only what the model already contains.

The efficient-thinking frontier — size vs. search. The core efficiency question, concretely: for a fixed compute budget, spend it on a bigger model or on more search over a

smaller one? Sweeping a clean size ladder (Qwen2.5 0.5B/1.5B/3B) × self-consistency N on GSM8K:

model	params	greedy	sc@4	sc@16
Qwen2.5-0.5B	0.5B	30.0%	25.0%	41.2%
Qwen2.5-1.5B	1.5B	48.8%	50.0%	58.8%
Qwen2.5-3B	3.0B	67.5%	76.2%	83.8%

On accuracy vs. compute (≈ params × N), **every small-model-plus-search point is dominated by a bigger model with less search**: 3B greedy (67.5%, compute ≈ 3) beats 0.5B@N=16 (41.2%, ≈ 8) *and* 1.5B@N=16 (58.8%, ≈ 24). In reasoning, **base-model size dominates search** on the efficiency frontier — the *opposite* of chess, where search on a fixed 3.45M net was the efficient lever. The reconciliation is the thesis itself: *search extracts what the model already contains*. The chess net was already strong on its task (~2150 open-loop), so search extracted a lot; these small LLMs are too weak on GSM8K for search to rescue — you cannot self-consistency your way up from a base that rarely finds the answer at all. **Search complements a competent base; it cannot substitute for one.** "Buy search, not size" holds only above a base-competence threshold. (Compute ≈ params × N is coarse, but the domination is large enough to survive it.)

5. Arm C — sequential control (a gridworld MDP)

To test the pattern in a *third modality* — sequential decision-making, neither a board game nor language — we use a stochastic 8×8 gridworld with known dynamics, so value iteration gives the exact optimal value V *(a perfect oracle). We hand the controller a* degraded *evaluator, V* plus Gaussian noise of scale σ (in units of the value spread), and vary the horizon h of an MPC-style lookahead (h=1 = greedy / open-loop; larger h = closed-loop search). Return is normalized so 0% = a random policy and 100% = optimal.

evaluator noise σ	open-loop (h=1)	h=2	h=3
0.0 (perfect)	100%	100%	100%
0.25	22%	**97%**	71%
0.5	33%	33%	43%
1.0	11%	28%	34%
2.0	16%	34%	34%

The same two findings recur. **With a perfect evaluator, open-loop is already optimal and search adds nothing** — search earns its cost only when the evaluator is imperfect.

With a mildly noisy evaluator (σ = 0.25), search compensates dramatically — the greedy policy collapses to 22% of optimal, but two-step lookahead recovers it to 97%: evaluator × search, in control. But **as the evaluator degrades further, search recovers less and less** (σ ≥ 1: lookahead reaches only ~30–34% and cannot recover optimal) — past a point the evaluator, not the search horizon, is the binding constraint. Decomposition and evaluator-bottleneck, both holding in a control/RL setting with an exact oracle.

6. Self-improvement — can the flywheel raise the evaluator with no external teacher?

Stated value-first: *fix the evaluator first* — distill better-than-current value/policy targets (Monte-Carlo rollouts / MCTS-backed, anchored on real terminal outcomes) back into the net; strength follows. We tested this five ways across two games. **It never broke the plateau — and why is the result.**

- **Connect-4, from scratch:** the evaluator improves (oracle value-MAE 0.48 → 0.355 with more search) and strength tracks it early (GELO +119 → ~+400), confirming *fix-the-evaluator-and-strength-follows* in miniature — but strength plateaus at **~+400** and **4× more search per move did not raise it**. More search budget is not the missing ingredient.
- **The plateau is a (near) seed-independent attractor:** warm-starting from the supervised +644 net does not preserve it — the first iteration erodes it to +189, then it recovers only to **~+480–500**, well below the +644 seed. Self-play pulls a *better* evaluator down toward its own signal quality (the same erosion as the chess self-learned test, 1214 → 833).
- **Chess, three negatives:** from scratch, open-loop Elo bounced 254–384 with no climb over 44 iters (~1,400 games) — a data-volume wall, not a method failure; warm-started from a weak-policy seed, it stalled at 300–326 (a **bootstrap barrier**: MCTS quality depends on the policy prior, so a near-random prior can't generate improving targets); and started from a genuinely self-learned 1214 plateau net, it **degraded to 833** — search on that value head doesn't play far enough above 1214 to produce improving targets.
- **An external oracle breaks the plateau (positive control).** Change *only* the value target — from the self-play outcome to an external depth-6 oracle, loop otherwise identical — and Connect-4 self-training **climbs past ~+400 to +719 by iteration 60** (MAE 0.36 → 0.31), toward the oracle's own level. The same loop that plateaus at +400 on self-generated signal climbs once the target carries external information. (Seed-independent: from the +644 supervised seed with oracle targets it reaches ~+676 — the *target source*, not the starting point, is what matters.)

Why it can't work for free. A Monte-Carlo rollout gives the true value of a position — but *under the current level of play* (the model plays both sides). Training on those labels

converges the evaluator to a perfectly-calibrated evaluator *of its own level*, capped there: no move better than the current level ever appears, so none can be learned. The only way rollouts push past the current level is to play them *above* it — with search — and then the per-iteration gain equals the **search-over-policy margin**, itself bounded by evaluator quality. This is AlphaZero's engine, and precisely why it needs deep search × millions of games: a large margin sustained over many iterations. At our scale the margin was too small. **Self-play cannot add information the evaluator doesn't already contain; raising the ceiling requires importing it.**

7. Discussion — what transfers, what shifts, what binds

1. **The decomposition transfers.** Strength = evaluator × search holds in two games, in language reasoning, and in sequential control, on one calibrated scale. Search is a large, portable lever that scales with inference compute and then saturates against the evaluator's ceiling (Elo-vs-sims in chess, accuracy-vs-N in reasoning, GELO-vs-MCTS in Connect-4, lookahead-vs-noise in the gridworld) — and it is worth nothing when the evaluator is already perfect (gridworld σ=0), everything when the evaluator is the only lever left.
2. **The lever balance shifts with spend.** Whichever currency is *starved* dominates returns: a thin evaluator (12k Connect-4 labels; a verifier-free consensus) makes search look all-important; feeding it (50k labels; a real verifier) rebalances. There is no domain-intrinsic split — only how much you have spent on each side. In reasoning the same logic flips the *frontier*: below a competence threshold, size (base capability) dominates search.
3. **Across the domains and compute budgets studied here, the evaluator is consistently the binding constraint**, shown independently: reasoning consensus is broken only by a better verifier (+14.2), and by a *graded* verifier (75 → 88); Connect-4 self-training is limited by value-head quality, not search budget; chess self-improvement stalls exactly when the evaluator is too weak for search to generate improving targets; and even the *judge* is evaluator-limited (search barely improves it).
4. **Self-improvement has a rate — the search-over-policy margin.** You cannot fix the evaluator for free: self-generated signal converges to the system's own level. Climbing requires search that plays above the current policy (bounded by the evaluator) or an external oracle that injects information. The positive proof is the flip side of every negative here — a perfect verifier unlocks +14.2 in reasoning; an oracle value target breaks the Connect-4 plateau; Stockfish labels carried chess to ~2800. *Training creates information; search extracts it; neither creates what an external oracle must supply.*

A vignette — capability per parameter, made vivid. Asked to play raw chess against the same Stockfish ladder as Efficient Thinking I, a frontier general LLM (Kimi-K2.5) scores ~56% vs a random mover, **0% vs Stockfish-1320**, and often cannot even emit a legal move — a performance rating of **≈341 GELO**. The 3.45M-parameter *specialist* from Paper I plays ~2150

open-loop and ~2800 with search. A tiny, correctly-shaped evaluator beats a giant generalist by **~1,800–2,500 GELO at the generalist's own game.** Scale is not what buys task capability; the right evaluator (and search over it) is. (Small sample, and part of the gap is the LLM's difficulty producing legal moves — but the order of magnitude is unambiguous.)

Related work

The evaluator-plus-search structure is the AlphaZero family [Silver et al. 2016; 2017; 2018] built on MCTS [Coulom 2006; Kocsis & Szepesvári 2006; Browne et al. 2012] and PUCT [Rosin 2011], with expert iteration [Anthony et al. 2017] as its self-play loop and KataGo [Wu 2019] as a strong open reference. The inference-time-compute view of reasoning — o1/R1 [OpenAI 2024; DeepSeek-AI 2025], self-consistency [Wang et al. 2023], tree-of-thought [Yao et al. 2023], STaR-style bootstrapping [Zelikman et al. 2022] — is the same lever at frontier scale; Jones [2021] documents test-time-compute scaling in board games. Our capability scale is the classical logistic latent-ability model shared by Elo, Bradley–Terry, and item-response theory (Rasch); pairwise LLM rating with a judge is the LMSYS-Arena approach [Zheng et al. 2023]. Benchmarks: GSM8K [Cobbe et al. 2021] and MATH [Hendrycks et al. 2021].

Reproducibility

Code and data generators for all three arms are in the repo. Control (`control/gridworld.py`): the MDP, exact value iteration, the noisy-evaluator × lookahead sweep — pure NumPy, exact oracle. Games (`games/`): Connect-4 bitboard engine + solver, the depth-limited alpha-beta oracle, the conv net, MCTS, the GELO calibrator (`c4_calibrate.py` , which prints the goodness-of-fit gate), and the self-play / oracle-value loops. Reasoning (`reasoning/`): the accuracy-vs-N sweep, the evaluator-quality ablation and gradient, the MATH IRT fit, the pairwise arena + master-judge (Bedrock), the judge-scaling test, and the size×search frontier. Large datasets and model weights are regenerable and not committed.

Appendix A — Experimental setup, per result

So the "what we did" is explicit and reproducible, the exact knobs behind each headline number:

Arm A — Connect-4 / GELO (`games/`). - *Opponent ladder & calibration* (`c4_calibrate.py` , `connect4_ab.py`): opponents are random plus depth-*d* alpha-beta (d = 1…6) against the perfect solver's move ordering. Ratings are fit by logistic MLE from the full round-robin **cross-table** (not per-opponent win-rates); the goodness-of-fit gate is mean |predicted − observed| pairwise win-rate = **0.058**; anchor **random := 0**. - *Evaluator net* (`c4_net.py`): a small conv policy+value network trained on **oracle-solver labels** (value =

game-theoretic value, policy = solver move distribution). Data-size sweep uses 12k / 24k / 50k label subsets. - *Placing an agent* (`place_agent`): the agent plays **24–40 games per rung** vs each ladder rung; the win-rate vector is fit to the ladder's GELO by the same logistic model. Open-loop = greedy on the policy head; closed-loop = **PUCT MCTS, 200 simulations**. - *Which-lever diagnostic* (`c4_diagnostic.py`): a data (3k–50k) × capacity (small ≈0.3M / large ≈4M params) grid, each cell placed both open- and closed-loop, to read where data / capacity / search each bind.

Arm B — reasoning (`reasoning/`), all Qwen2.5-Instruct-4bit under MLX. - *Self-consistency & oracle-best-of-N* (`reason_math_sweep.py`): **Qwen-4B**, GSM8K test, **120 problems**, temperature **0.8**, **1024**-token budget, **N ∈ {1,4,16,32}**; final answer via `#### <number>` regex; oracle-best-of-N = "is any of the N correct vs gold." - *Graded verifier* (same samples): a synthetic selector with per-item accuracy **q ∈ {0.5…1.0}**. - *Kimi-best-of-N* (`reason_bestofn.py`): **Qwen-1.5B**, **60 problems**, temp 0.8, **N ∈ {2,4,8}**; selector = **Kimi-K2.5** (Bedrock, temp 0) picking the correct candidate index — blinded to the gold. - *Judge scaling* (`reason_arena.py` judge, majority over repeats): Kimi judgments aggregated over **N ∈ {1…5}**; agreement measured against the ground-truth verifier on decisive pairs. - *Serial axis* (`reason_serial.py`): **greedy** (temp 0), max-tokens ∈ **{128,256,512,1024,2048}**, Qwen-1.5B and 3B. - *Size × search frontier*: Qwen **{0.5B,1.5B,3B}**, GSM8K, greedy + sc@{4,16}; compute proxy ≈ params × N. - *Pairwise reasoning-GELO* (`reason_arena.py`): **5 models** on **50 MATH problems**; **every** ordered pair scored by the Kimi-K2.5 judge (**blinded, answer-order randomized**); ratings by Bradley–Terry MLE (`bt_elo`), anchor **Kimi := 2800**; the verifier cross-check gives the 72% judge-agreement figure. - *Difficulty-anchored GELO* (`reason_gelo_irt.py`): the same models vs MATH difficulty tiers L1–L5, Rasch (1-parameter IRT) fit.

Arm C — control (`control/gridworld.py`). 8×8 gridworld, slip 0.1, γ = 0.95, scattered pits; exact **V*** by value iteration; degraded evaluator = V* + σ·(value-spread)·$\mathcal{N}(0,1)$; MPC lookahead **h ∈ {1,2,3}** (h Bellman backups, then greedy). Return = mean discounted reward over **400 episodes**, normalized to [random = 0, optimal = 100].

§6 self-improvement. Connect-4 (`c4_selfplay.py`): self-play games each iteration; value target = game **outcome** (baseline) vs **depth-6 oracle value** (`--oracle-value` , the positive control), loop otherwise identical; strength re-placed vs the ladder every iteration. Chess uses the analogous loop from three seeds (from-scratch / weak-policy warm-start / a self-learned +1214 net).

Infrastructure & honest limits. Two Apple-Silicon machines: **MLX / mlx-lm** for all Qwen inference and net training; **AWS Bedrock** (`moonshotai.kimi-k2.5` , us-east-1) for the master judge and the frontier-LLM chess vignette. Samples are deliberately modest (50–120 problems, 24–40 games/rung), so we report **effect sizes and recurring patterns, not tight confidence intervals**; where a result is within noise (e.g. the mid-model GELO bunching) we say so.

References

- Anthony, T., Tian, Z. & Barber, D. (2017). *Thinking Fast and Slow with Deep Learning and Tree Search*. NeurIPS.
- Browne, C. et al. (2012). *A Survey of Monte Carlo Tree Search Methods*. IEEE TCIAIG.
- Cobbe, K. et al. (2021). *Training Verifiers to Solve Math Word Problems* (GSM8K). arXiv:2110.14168.
- Coulom, R. (2006). *Efficient Selectivity and Backup Operators in Monte-Carlo Tree Search*. Computers and Games.
- DeepSeek-AI (2025). *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via RL*. arXiv:2501.12948.
- Hendrycks, D. et al. (2021). *Measuring Mathematical Problem Solving with the MATH Dataset*. NeurIPS.
- Jones, A. L. (2021). *Scaling Scaling Laws with Board Games*. arXiv:2104.03113.
- Kocsis, L. & Szepesvári, C. (2006). *Bandit Based Monte-Carlo Planning* (UCT). ECML.
- OpenAI (2024). *Learning to Reason with LLMs* (o1).
- Rosin, C. D. (2011). *Multi-Armed Bandits with Episode Context* (PUCT). Ann. Math. AI.
- Silver, D. et al. (2016; 2017; 2018). *Mastering Go / Go without Human Knowledge / a General RL Algorithm* (AlphaGo, AlphaGo Zero, AlphaZero). Nature; Nature; Science.
- Wang, X. et al. (2023). *Self-Consistency Improves Chain-of-Thought Reasoning*. ICLR.
- Wu, D. J. (2019). *Accelerating Self-Play Learning in Go* (KataGo). arXiv:1902.10565.
- Yao, S. et al. (2023). *Tree of Thoughts*. NeurIPS.
- Zelikman, Y. et al. (2022). *STaR: Bootstrapping Reasoning with Reasoning*. NeurIPS.
- Zheng, L. et al. (2023). *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. NeurIPS.
- **Software/data:** Stockfish · MLX · mlx-lm · AWS Bedrock (Kimi-K2.5) · Lichess cloud evals · HuggingFaceH4/MATH-500.